Lecture 2: Vector Spaces in Imaging

Seonyeong Park, Ph.D. (📧 : sp33@illinois.edu)
Research Assistant Professor
Bioengineering
University of Illinois Urbana-Champaign
January 22, 2025

🔑 Learning Objectives

By the end of this lecture, you should be able to understand:

- Linear vector spaces
- Hilbert spaces
- The $\mathbb{L}_2$ norm

📌 Before We Begin

Vectors can exist in both *discrete* and *continuous* forms. To make the concepts easier to grasp, the *discrete* version of the material will be explained first. Then, the corresponding *continuous* version and its relationship to the discrete form will be discussed.

📝 Example

Image $\hat{\mathbf{f}}$ representing the form of an object using a ***finite number of values***

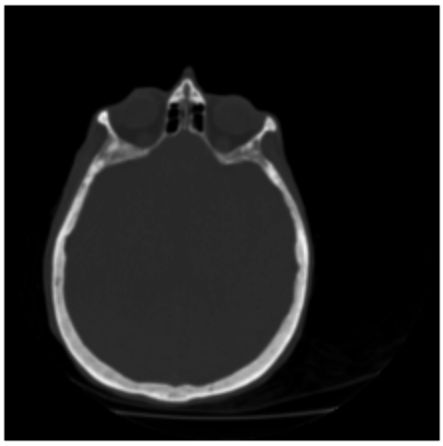
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from skimage.data import brain

# Load a stack of brain images
image_stack = brain()

# Select one brain image slice
f_hat = image_stack[3]

# Plot f_hat
plt.imshow(f_hat, cmap='gray')
```

```
plt.axis('off')
plt.show()
```



In [2]:
```
# Print type and dimensionality of f_hat
print(f"Type of f_hat: {type(f_hat)}")
print(f"Number of dimensions: {f_hat.ndim}")
```

```
Type of f_hat: <class 'numpy.ndarray'>
Number of dimensions: 2
```

What is an `Array`?

A collection of elements indexed in a structured manner (this term is commonly used in **programming**.)

Equivalent Concepts in Mathematics and Physics

- `Vectors`
- `Matrices`
- `Tensors`

Representation of `Vectors`, `Matrices`, and `Tensors` Using `Arrays`

- `Vector`: 1D array
- `Matrix`: 2D array
- `Tensor`: N-dimensional (ND) array ($N \geq 3$)

Mathematical Notations

Number Type	`Vector`	`Matrix`	`Tensor` (3D Array Example)
Natural numbers	$\hat{\mathbf{f}} \in \mathbb{N}^{M}$	$\hat{\mathbf{f}} \in \mathbb{N}^{M \times L}$	$\hat{\mathbf{f}} \in \mathbb{N}^{M \times L \times K}$

Number Type	`Vector`	`Matrix`	`Tensor` (3D Array Example)
Integers	$\hat{\mathbf{f}} \in \mathbb{Z}^{M}$	$\hat{\mathbf{f}} \in \mathbb{Z}^{M \times L}$	$\hat{\mathbf{f}} \in \mathbb{Z}^{M \times L \times K}$
Real numbers	$\hat{\mathbf{f}} \in \mathbb{R}^{M}$	$\hat{\mathbf{f}} \in \mathbb{R}^{M \times L}$	$\hat{\mathbf{f}} \in \mathbb{R}^{M \times L \times K}$
Complex numbers	$\hat{\mathbf{f}} \in \mathbb{C}^{M}$	$\hat{\mathbf{f}} \in \mathbb{C}^{M \times L}$	$\hat{\mathbf{f}} \in \mathbb{C}^{M \times L \times K}$

Numbers in Digital Systems

- `int8`, `int16`, and `int32`: Fixed-width integers ($\mathbb{Z}$) with varying ranges based on bit width
- `float16`, `float32`, and `float64`: Floating-point approximations of real numbers ($\mathbb{R}$) with varying precision and range

In [3]:
```
# Print data type of f_hat
print(f"Data type of f_hat: {f_hat.dtype}")
```

```
Data type of f_hat: uint16
```

❓ Referring back to the image plotted above, what is the mathematical expression for the image as a matrix $\hat{\mathbf{f}}$?

In [4]:
```
# Print shape of f_hat
print(f"Shape of f_hat: {f_hat.shape}")
```

```
Shape of f_hat: (256, 256)
```

Given a 2D array with a shape of (256, 256) with a data type of `uint16`,

$$\hat{\mathbf{f}} \in \mathbb{Z}^{256 \times 256}$$

❓ What is the mathematical expression for the image as a vector $\hat{\mathbf{f}}$?

In [5]:
```
# Convert a matrix to a vector
# f_hat_vec = f_hat.ravel()
f_hat_vec = f_hat.flatten()

# Print type, data type, and shape
print(f"Shape of f_hat_vec: {f_hat_vec.shape}")
```

```
Shape of f_hat_vec: (65536,)
```

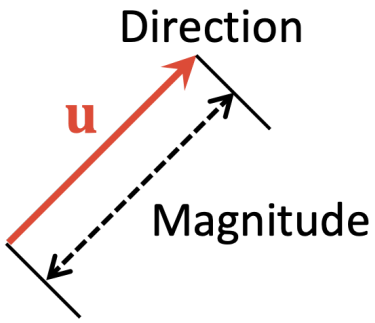
Given a 1D array with a shape of (65536,) with a data type of `uint16`,

$$\hat{\mathbf{f}} \in \mathbb{Z}^{65536}$$

Vector

Quantity that has both *magnitude (scalar)* and *direction*

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} u_1 & u_2 & \dots & u_N \end{bmatrix}^T = (u_1, u_2, \dots, u_N)$$



- $\mathbf{u}$: Vector
- u_n: Vector component (`u[n-1]`)

💡 *Why `n-1`? Because NumPy array indices start at 0.*

In [6]:
```
# Define vector u
u = np.array([1.2, 1])

# Vector magnitude
u_magnitude = np.sqrt(np.sum(np.square(u)))

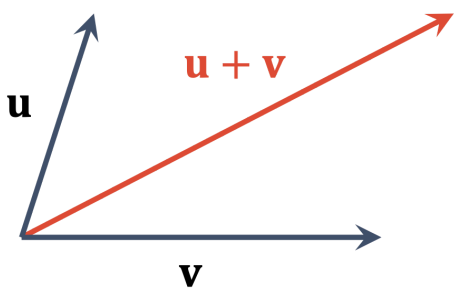
# Vector direction
u_direction = u / u_magnitude

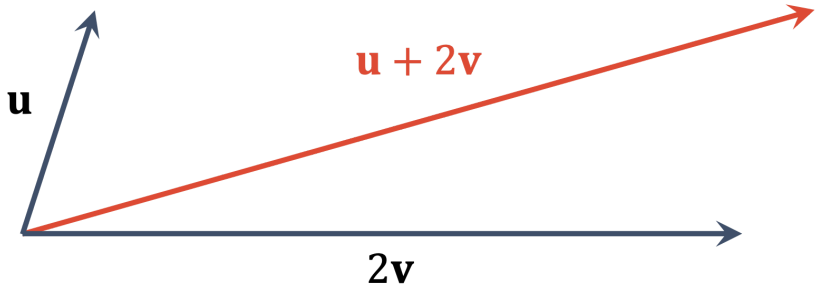
# Print vector magnitude and direction
print(f"Magnitude of u: {u_magnitude}")
print(f"Direction of u: {u_direction}")
```

```
Magnitude of u: 1.5620499351813308
Direction of u: [0.76822128 0.6401844 ]
```

Linear Vector Space

A collection of vectors that may be ***added together*** and ***multiplied by a scalar*** according to a set of rules or axioms





In [7]:
```
# Define vector v
v = np.array([2, 0])
```

```
# Add together
u_plus_v = u + v

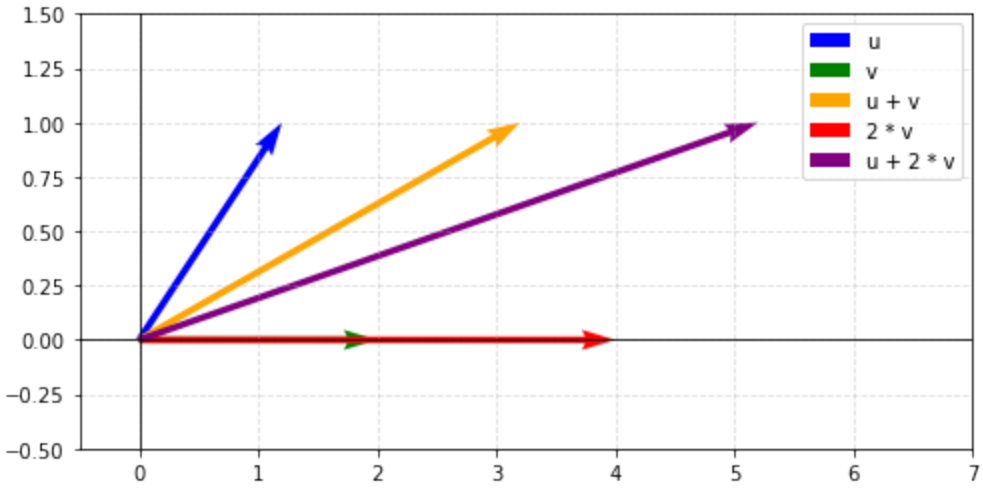
# Multiply by a scalar
v_times_2 = 2 * v

# Add together and multiply by a scalar
u_plus_v_times_2 = u + 2 * v

# Print vectors
print(f"u: {u}")
print(f"v: {v}")
print(f"u + v: {u_plus_v}")
print(f"2 * v: {v_times_2}")
print(f"u + 2 * v: {u_plus_v_times_2}")
```

```
u: [1.2 1. ]
v: [2 0]
u + v: [3.2 1. ]
2 * v: [4 0]
u + 2 * v: [5.2 1. ]
```

In [8]:
```
# Plot vectors
plt.figure(figsize=(8, 4))
ax = plt.gca()
ax.quiver(0, 0, u[0], u[1], angles='xy', scale_units='xy', scale=1, color='blue', label=
ax.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1, color='green', label
ax.quiver(0, 0, u_plus_v[0], u_plus_v[1], angles='xy', scale_units='xy', scale=1, color=
ax.quiver(0, 0, v_times_2[0], v_times_2[1], angles='xy', scale_units='xy', scale=1, colo
ax.quiver(0, 0, u_plus_v_times_2[0], u_plus_v_times_2[1], angles='xy', scale_units='xy',
plt.xlim(-0.5, 7)
plt.ylim(-0.5, 1.5)
plt.grid(True, linestyle='--', alpha=0.4)
plt.axhline(0, color='black', linewidth=0.8)  # x-axis
plt.axvline(0, color='black', linewidth=0.8)  # y-axis
plt.legend(loc='upper right')
plt.show()
```



Axioms of Linear Vector Spaces

- **Addition**
  - Commutative property: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
  - Associative property: $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$
- **Scalar multiplication** ($a \in \mathbb{C}$ and $b \in \mathbb{C}$)
  - Commutative property: $a(b\mathbf{u}) = b(a\mathbf{u})$
  - Distributive property
    - With respect to scalar addition: $(a + b)\mathbf{u} = a\mathbf{u} + b\mathbf{u}$
    - With respect to vector addition: $a(\mathbf{u} + \mathbf{v}) = a\mathbf{u} + a\mathbf{v}$
- Existence of an identity operator for scalar multiplication: $1\mathbf{u} = \mathbf{u}$
- Existence of a zero vector: $\exists \mathbf{0} | \mathbf{u} + \mathbf{0} = \mathbf{u}$
- Existence of an inverse element: $\exists -\mathbf{u} | \mathbf{u} + (-\mathbf{u}) = \mathbf{0}$

💡 *For some problems, it might be useful to define new arithmetic, numerical operations.*

❓ Assume you have vectors $\mathbf{u}$, $\mathbf{v}$, and $\mathbf{w}$ representing images of the same size. How would you measure their similarity?

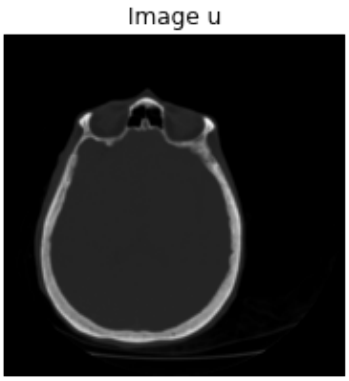
In [9]:
```
# Define matrices u, v, and w
u_mat = image_stack[0]
v_mat = image_stack[1]
w_mat = image_stack[5]
```

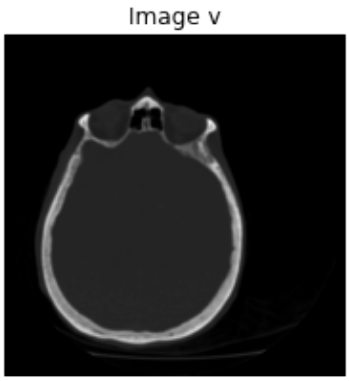
In [10]:
```
# Plot images u, v, and w
plt.figure(figsize=(12, 3))

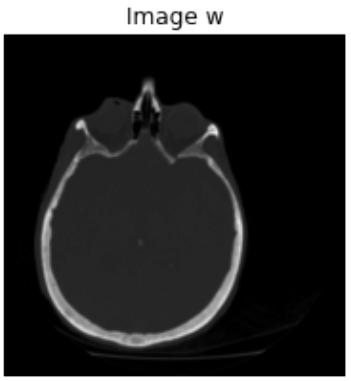
plt.subplot(1, 3, 1)
plt.imshow(u_mat, cmap='gray')
plt.title('Image u')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(v_mat, cmap='gray')
plt.title('Image v')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(w_mat, cmap='gray')
plt.title('Image w')
plt.axis('off')
plt.tight_layout()
plt.show()
```







In [11]:
```
# Convert the matrices u, v, and w to vectors
u_vec = u_mat.flatten()
v_vec = v_mat.flatten()
w_vec = w_mat.flatten()
```

Distance (Metric)

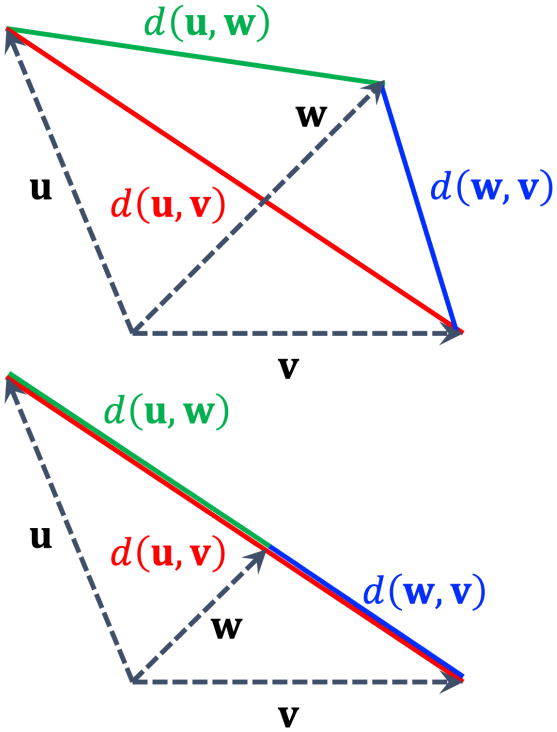
The distance between $\mathbf{u} \in \mathbb{C}^N$ and $\mathbf{v} \in \mathbb{C}^N$, denoted by $d(\mathbf{u}, \mathbf{v})$, satisfies the following conditions:

$$d(\mathbf{u}, \mathbf{v}) = d(\mathbf{v}, \mathbf{u})$$

$$d(\mathbf{u}, \mathbf{v}) \geq 0$$

$$d(\mathbf{u}, \mathbf{v}) = 0 \text{ if only if } \mathbf{u} = \mathbf{v}$$

$$d(\mathbf{u}, \mathbf{v}) \leq d(\mathbf{u}, \mathbf{w}) + d(\mathbf{w}, \mathbf{v}) \quad \textbf{Triangle inequality}$$

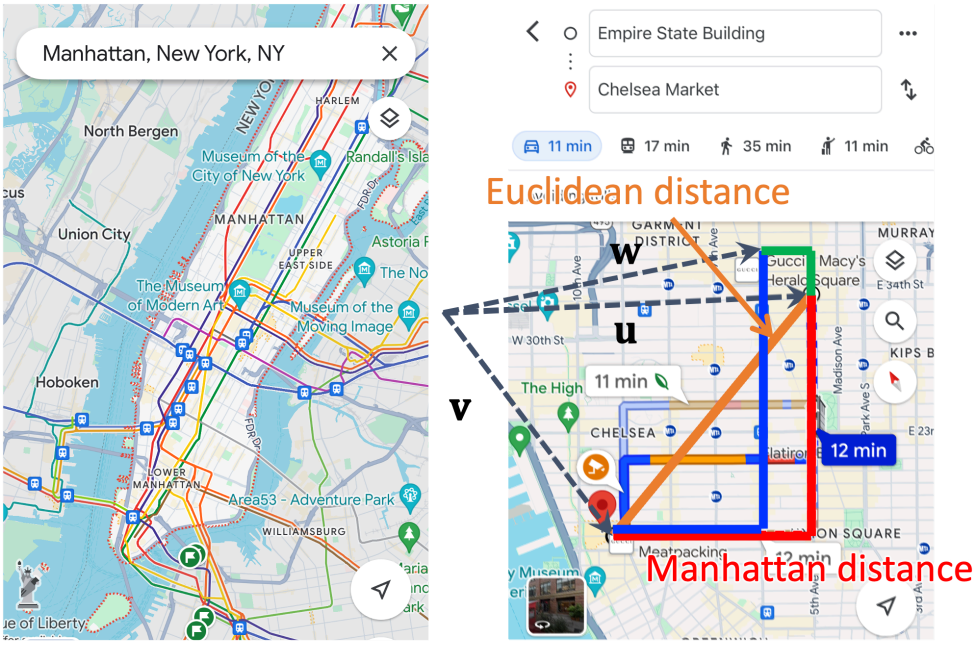


Euclidean Distance

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2 + \cdots + (u_N - v_N)^2}$$

Manhattan Distance

$$d(\mathbf{u}, \mathbf{v}) = |u_1 - v_1| + |u_2 - v_2| + \cdots + |u_N - v_N|$$



Images from Google map

In [12]:
```
# Calculate the Euclidean distance
dist_euc_u_v = np.sqrt(np.sum(np.square(u_vec - v_vec)))
dist_euc_u_w = np.sqrt(np.sum(np.square(u_vec - w_vec)))

# Calculate the Manhattan distance
dist_man_u_v = np.sum(np.abs(u_vec - v_vec))
dist_man_u_w = np.sum(np.abs(u_vec - w_vec))

# Print the distances
print(f"Euclidean distance between vectors u and v: {dist_euc_u_v}")
print(f"Euclidean distance between vectors u and w: {dist_euc_u_w}")
print(f"Manhattan distance between vectors u and v: {dist_man_u_v}")
print(f"Manhattan distance between vectors u and w: {dist_man_u_w}")
```

```
Euclidean distance between vectors u and v: 25153.01212578724
Euclidean distance between vectors u and w: 27542.972461228655
Manhattan distance between vectors u and v: 1071463009
Manhattan distance between vectors u and w: 1121024392
```

In the example above, $\mathbf{v}$ is more similar to $\mathbf{u}$ than $\mathbf{w}$ is.

Norm (Vector Size/Magnitude/Length)

Norm of $\mathbf{u}$: The distance between $\mathbf{u}$ and the origin, $\|\mathbf{u}\| = d(\mathbf{u}, \mathbf{0})$

$$\|\mathbf{u}\| \geq 0$$

$$\|\mathbf{u}\| = 0 \text{ if and only if } \mathbf{u} = \mathbf{0}$$

$$\|a\mathbf{u}\| = |a|\|\mathbf{u}\|$$

$$\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$$

Possible Definitions of the Norm

$\mathbb{L}_1$ norm: $\|\mathbf{u}\|_1 = \sum_{n=1}^{N} |u_n|$

$\mathbb{L}_2$ norm: $\|\mathbf{u}\|_2 = \sqrt{\sum_{n=1}^{N} |u_n|^2}$

$\mathbb{L}_p$ norm: $\|\mathbf{u}\|_p = [\sum_{n=1}^{N} |u_n|^p]^{1/p}$, where $p \in \mathbb{R} \geq 0$

Norm and Distance between $\mathbf{u}$ and $\mathbf{v}$

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|$$

- Manhattan distnace: $\|\mathbf{u} - \mathbf{v}\|_1$
- Euclidean distance: $\|\mathbf{u} - \mathbf{v}\|_2$

❓ Calculate the $\mathbb{L}_1$ and $\mathbb{L}_2$ norms of the vector representing the first slice of the brain image stack `skimage.data.brain` using: 1) the NumPy mathematical functions `numpy.sum`, `numpy.abs`, `numpy.sqrt`, and `numpy.square` and 2) the NumPy linear algebra function `numpy.linalg.norm`.

In [13]:
```
# Calculate L1 norm of u
l1_norm = np.sum(np.abs(u))
l1_norm_linalg = np.linalg.norm(u, ord=1) # Using NumPy linear algebra function

# Calculate L2 norm of u
l2_norm = np.sqrt(np.sum(np.square(u)))
l2_norm_linalg = np.linalg.norm(u, ord=2) # Using NumPy linear algebra function

# Print u and its L1 and L2 norms
print(f"u: {u}")
print(f"L1 norm of u:                {l1_norm}")
print(f"L1 norm of u (NumPy linalg): {l1_norm_linalg}")
print(f"L2 norm of u:                {l2_norm}")
print(f"L2 norm of u (NumPy linalg): {l2_norm_linalg}")
```

```
u: [1.2 1. ]
L1 norm of u:                2.2
L1 norm of u (NumPy linalg): 2.2
L2 norm of u:                1.5620499351813308
L2 norm of u (NumPy linalg): 1.5620499351813308
```

Inner Product (Scalar Product)

The inner product between $\mathbf{u} \in \mathbb{C}^N$ and $\mathbf{v} \in \mathbb{C}^N$ is defined as:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \sum_{n=1}^{N} u_n^* v_n,$$

where u_n^* is the complex conjugate of the complex number u_n.

The inner product, denoted by $\langle \mathbf{u}, \mathbf{v} \rangle$, satisfies the following conditions:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \langle \mathbf{v}, \mathbf{u} \rangle^*$$

$$\langle \mathbf{u}, a\mathbf{v} \rangle = a\langle \mathbf{u}, \mathbf{v} \rangle, \text{ where } a \in \mathbb{C}$$

$$\langle a\mathbf{u}, \mathbf{v} \rangle = a^*\langle \mathbf{u}, \mathbf{v} \rangle$$

$$\langle \mathbf{u} + \mathbf{v}, \mathbf{w} \rangle = \langle \mathbf{u}, \mathbf{w} \rangle + \langle \mathbf{v}, \mathbf{w} \rangle$$

$$\langle \mathbf{u}, \mathbf{u} \rangle \geq 0$$

$$\langle \mathbf{u}, \mathbf{u} \rangle = 0 \text{ if and only if } \mathbf{u} = \mathbf{0}$$

Dot Product

A specific type of the inner product defined for finite-dimensional (real or complex) Euclidean vectors

The dot product of two real Euclidean vectors $\mathbf{u} \in \mathbb{E}^N$ and $\mathbf{v} \in \mathbb{E}^N$ is given by:

$$\langle \mathbf{u}, \mathbf{v} \rangle = u_1 v_1 + u_2 v_2 + \cdots + u_N v_N = \|\mathbf{u}\| \|\mathbf{v}\| \cos\theta,$$

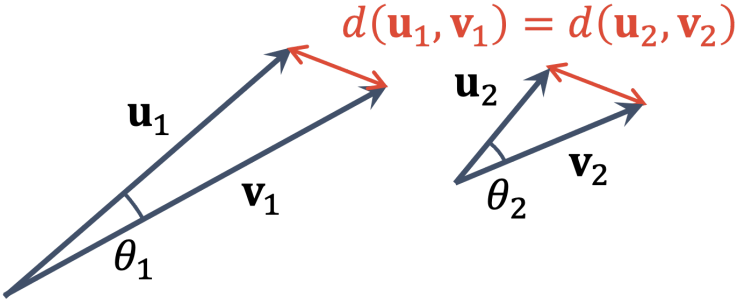
where θ is the angle between $\mathbf{u}$ and $\mathbf{v}$.

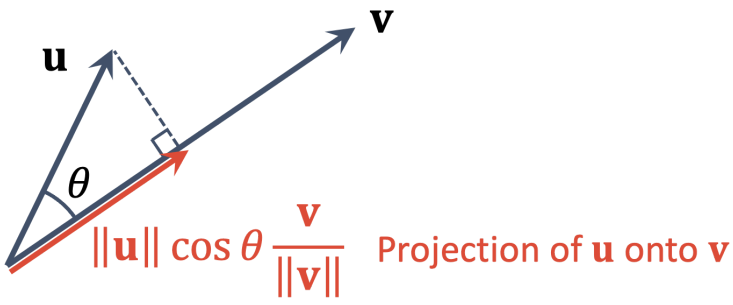
Cosine Similarity

$$\cos\theta = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

Projection of $\mathbf{u}$ onto $\mathbf{v}$

$$\text{proj}_{\mathbf{v}}\mathbf{u} = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{v}\|} \frac{\mathbf{v}}{\|\mathbf{v}\|} = \|\mathbf{u}\| \cos\theta \frac{\mathbf{v}}{\|\mathbf{v}\|}$$



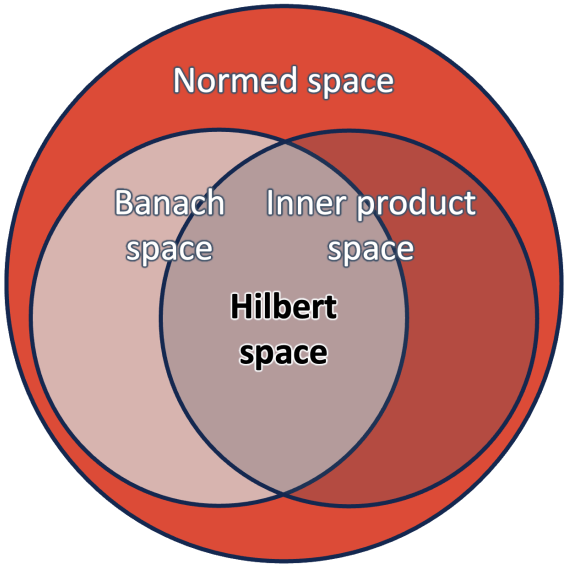


$\mathbb{L}_2$ Norm and Inner Product

$$\|\mathbf{u}\|_2 = \sqrt{\langle \mathbf{u}, \mathbf{u} \rangle} = \sqrt{\sum_{n=1}^{N} u_n^* u_n}$$

Hilbert Spaces

A special case of a ***complete***, ***linear*** vector space, where a ***norm*** and an ***inner product*** are defined



$$\text{Hilbert space} \subseteq \text{Inner product space} \subseteq \text{Normed space} \subseteq \text{Linear vector space}$$
$$\text{Hilbert space} \subseteq \text{Banach space} \subseteq \text{Normed space} \subseteq \text{Linear vector space}$$
$$\text{Hilbert space} = \text{Inner product space} \cap \text{Banach space}$$

	Linearity	Norm	Inner Product	Completeness
Linear vector space	O	X	X	X
Normed space	O	$\mathbb{L}_p$	X	X
Inner product space	O	$\mathbb{L}_2$	O	X
Banach space	O	$\mathbb{L}_p$	X	O
Hilbert space	O	$\mathbb{L}_2$	O	O

💡 *The space of most use in this lecture is Hilbert space.*

💡 *An important example of a Hilbert space is the N-dimensional Euclidean space $\mathbb{E}^N$, where vectors can have either real- or complex-valued components.*

Extending to the Continuous Form

Vector

- Discrete form: $\mathbf{f} = \{f_n, n = 1, 2, \ldots\}$

    - ***An ordered set of N (real or complex) numbers*** (N-tuple), where the numbers (components) are **indexed by a discrete variable**
        - *Finite discrete* vectors (**Euclidean vectors**)
        - *(Countably) infinite discrete* vectors

- Continuous form: $\mathbf{f} = f(x), x \in (\alpha, \beta)$
    - ***A function*** whose components (values of the function) are **indexed by a continuous variable** (e.g., $x \in (\alpha, \beta)$ and $t \in [0, T]$)

Inner Product (Scalar Product)

- Discrete form:

$$\langle \mathbf{f}_1, \mathbf{f}_2 \rangle = \sum_{n=1}^{N} f_{1n}^* f_{2n}$$

- Continuous form:

$$\langle \mathbf{f}_1, \mathbf{f}_2 \rangle = \int_{\alpha}^{\beta} dx \, f_1^*(x) \, f_2(x)$$

💡 *Discrete sums* ($\sum_{n=1}^{N}$) *correspond to continuous integrals* ($\int_{\alpha}^{\beta} dx$).

Hilbert Spaces

- Discrete example: Euclidean space $\mathbb{E}^N$
    - The space of ordered sets of N complex numbers $\{f_n, n = 1, \ldots, N\}$, $f_n \in \mathbb{C}$, where the inner product is defined as: $\langle \mathbf{f}_1, \mathbf{f}_2 \rangle = \sum_{n=1}^{N} f_{1n}^* f_{2n}$

- Continuous example 1: $\mathbb{L}_2(\alpha, \beta)$, also denoted $\mathbb{L}_2$
    - The space of complex-valued functions $f(x)$, where the inner product is defined as: $\langle \mathbf{f}_1, \mathbf{f}_2 \rangle = \int_{\alpha}^{\beta} dx \, f_1^*(x) \, f_2(x)$
    - The space of square-integrable functions over the range (α, β)

💡 $\mathbb{L}_2$ *norm of a discrete vector:* $\|\mathbf{u}\|_2 = \sqrt{\sum_{n=1}^{N} |u_n|^2}$ *and that of a continuous vector:* $\|\mathbf{u}\|_2 = \sqrt{\int_{\alpha}^{\beta} dx \, |u(x)|^2}$

- Continuous example 2: $\mathbb{L}_2(\mathbb{R}^2)$, also denoted $\mathbb{L}_2[(-\infty, \infty) \times (-\infty, \infty)]$
    - The space of complex-valued functions $f(\mathbf{r}), \mathbf{r} = (x, y) \in \mathbb{R}^2$, where inner products are defined by 2D integrals: $\langle \mathbf{f}_1, \mathbf{f}_2 \rangle = \int d\mathbf{r} \, f_1^*(\mathbf{r}) \, f_2(\mathbf{r})$
    - The space of square-integrable functions in $\mathbb{R}^2$

	Discrete Form	Continuous Form
Vector	$\mathbf{f} = \{f_n, n = 1, 2, \ldots\}$	$\mathbf{f} = f(x), x \in (\alpha, \beta)$
Inner Product	$\langle \mathbf{f}_1, \mathbf{f}_2 \rangle = \sum_{n=1}^{N} f_{1n}^* f_{2n}$	$\langle \mathbf{f}_1, \mathbf{f}_2 \rangle = \int_{\alpha}^{\beta} dx\, f_1^*(x)\, f_2(x)$
Hilbert Spaces	$\mathbb{E}^N$	$\mathbb{L}_2(\alpha, \beta)$, also denoted $\mathbb{L}_2$
		$\mathbb{L}_2(\mathbb{R}^2)$, also denoted $\mathbb{L}_2[(-\infty, \infty) \times (-\infty, \infty)]$

❓ Between the discrete form and the continuous form of vectors, which one do the brain images utilized above correspond to?

❓ Recall an object to be imaged, measurements, and a reconstructed image in the pipeline of imaging in medicine. These can be represented by vectors. Which form does each correspond to? Explain why.

❓ In Python, finite-dimensional vectors are easily defined using the `NumPy` library, as shown in the examples above. How would you write code to define continuous vectors?

❓ Write code to define a vector $\mathbf{f}$ representing a uniform square object. The squre has a size of 8 mm and is centered at (0, 0) mm. The value inside the square is 1.

In [14]:
```python
# Define a function f(r) representing uniform square object
# r = (x, y) \in R^2
def f(r):
  val = 1
  size = 8 # [mm]
  center = np.array([0, 0]) # [mm]

  if np.all(np.abs(r - center) <= size/2.):
    return val
  else:
    return 0
```

❓ Represent the Hilbert space of the vector $\mathbf{f}$, corresponding to the uniform square object described above, using mathematical notation.